

**ON FORMAL VERIFICATION TECHNIQUES FOR MASKED
CIRCUITS: APPROACHES AND THEORETICAL
UNDERPINNINGS**

by
KAĞAN GÜLSÜM

Submitted to the Graduate School of Engineering and Natural Sciences
in partial fulfilment of
the requirements for the degree of Master of Science
in Computer Science and Engineering

Sabancı University
July 2026

**ON FORMAL VERIFICATION TECHNIQUES FOR MASKED
CIRCUITS: APPROACHES AND THEORETICAL
UNDERPINNINGS**

Approved by:

Asst. Prof. SÜHA ORHUN MUTLUERGİL
(Thesis Supervisor)

Prof. ERKAY SAVAŞ

Asst. Prof. KLAUS FREIHERR VON GLEISSENTHAL

Date of Approval: July 22, 2026

KAĞAN GÜLSÜM 2026 ©

All Rights Reserved

ABSTRACT

ON FORMAL VERIFICATION TECHNIQUES FOR MASKED CIRCUITS: APPROACHES AND THEORETICAL UNDERPINNINGS

KAĞAN GÜLSÜM

Computer Science and Engineering, M.Sc. Thesis, July 2026

Thesis Supervisor: Asst. Prof. SÜHA ORHUN MUTLUERGİL

Keywords: keyword1, keyword2, keyword3, keyword4

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum. Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas tempus, tellus eget condimentum rhoncus, sem quam semper libero, sit amet adipiscing sem neque sed ipsum. Nam quam nunc, blandit vel, luctus pulvinar, hendrerit id, lorem. Maecenas nec odio et ante tincidunt tempus. Donec vitae sapien ut libero venenatis faucibus. Nullam quis ante. Etiam sit amet orci eget eros faucibus tincidunt. Duis leo. Sed fringilla mauris sit amet nibh. Donec sodales sagittis magna. Sed consequat, leo eget bibendum sodales, augue velit cursus nunc, Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. et magnis dis parturient montes, nascetur ridiculus mus.

ÖZET

MASKELENMİŞ DEVRELER İÇİN FORMAL DOĞRULAMA TEKNİKLERİ ÜZERİNE: YAKLAŞIMLAR VE KURAMSAL TEMELLER

KAĞAN GÜLSÜM

Bilgisayar Bilimi ve Mühendisliği, Yüksek Lisans Tezi, Temmuz 2026

Tez Danışmanı: Dr. Öğr. Üyesi SÜHA ORHUN MUTLUERGİL

Anahtar Kelimeler: kelime1, kelime2, kelime3, kelime4

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum. Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas tempus, tellus eget condimentum rhoncus, sem quam semper libero, sit amet adipiscing sem neque sed ipsum. Nam quam nunc, blandit vel, luctus pulvinar, hendrerit id, lorem. Maecenas nec odio et ante tincidunt tempus. Donec vitae sapien ut libero venenatis faucibus. Nullam quis ante. Etiam sit amet orci eget eros faucibus tincidunt. Duis leo. Sed fringilla mauris sit amet nibh. Donec sodales sagittis magna. Sed consequat, leo eget bibendum sodales, augue velit cursus nunc, Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. et magnis dis parturient montes, nascetur ridiculus mus.

ACKNOWLEDGEMENTS

I would like to thank my advisor and family.

Dedicated to Okan Buruk...

TABLE OF CONTENTS

ABSTRACT	iv
ÖZET	v
LIST OF TABLES	xii
LIST OF FIGURES	xiii
LIST OF SYMBOLS	xiv
LIST OF ABBREVIATIONS	xv
1. INTRODUCTION	1
1.1. Side-Channel Attacks and the Masking Countermeasure.....	1
1.2. Verifying Masked Implementations	1
1.3. Scope and Threat Model	1
1.4. Contributions.....	2
1.5. Thesis Outline	2
2. PRELIMINARIES	3
2.1. Notation and Mathematical Background	3
2.1.1. The Field $\mathbb{F}_2$ and Multilinear Boolean Polynomials.....	3
2.1.2. Probability Distributions and Statistical Independence.....	3
2.1.3. Boolean Circuits and Gates	3
2.2. Secret Sharing and Masking.....	5
2.2.1. Boolean Masking	5
2.3. The ISW Probing Model	5
2.3.1. t -Threshold Probing and Observation Sets.....	6
2.3.2. d -Probing Security	6
2.3.3. Probabilistic Non-Interference	6
2.3.4. Threshold and Strong Non-Interference	6

2.4. Randomness Substitution	6
2.5. Computational Hardness Preview	8
3. RELATED WORK	9
3.1. Language-Based Verification	9
3.1.1. Verified Proofs of Higher-Order Masking	9
3.1.2. Large Sets and the Exploration Algorithm.....	9
3.1.3. Witness Replay and Derivation Reuse	10
3.1.4. MaskVerif With Physical Defaults	10
3.2. Fourier-Analytic Verification	10
3.2.1. The Xiao-Massey Criterion and Approximations	10
3.2.2. SAT-Based Realization and Its Cost	10
3.3. Other Approaches	11
3.3.1. Model Counting and SMT	11
3.3.2. Compositional Reasoning and Relabeling	11
3.4. Position of This Work	11
4. COUPLINGS AND PROBING SECURITY	12
4.1. The Counting Problem Is #P-Complete	12
4.1.1. From Independence to Distribution Counting	12
4.1.2. #P-Completeness and Its Consequences for Verification.....	12
4.2. Couplings of Probabilistic Programs	13
4.2.1. Coupling Two Runs Under Different Secrets	13
4.2.2. Coupling Functions: Randoms to Randoms.....	13
4.3. Probing Security via Couplings	13
4.3.1. d -Probing Security Implies a (Family of) Coupling Functions ..	13
4.3.2. Secret-Indistinguishability	14
4.4. Substitutions as a Coupling Construction	14
4.4.1. The Derivation of an Explicit Coupling Witness.....	14
4.4.2. Why a Successful Rewrite Yields a Valid Coupling	14
5. DESIGN OF XXX	15
5.1. Pipeline Overview	15
5.2. Component I: The Rewrite Engine	15
5.2.1. The Simple Rewrite Rule	16

5.2.2. Fallback: General Substitution.....	18
5.2.3. Relation to the Coupling-Construction View	19
5.3. Component II: Traversing the Subset Space.....	22
5.3.1. The Representation of the Subset Space	22
5.3.2. Large-Set Pruning.....	23
5.3.3. Completeness via Vandermonde’s Identity	24
5.4. Component III: Extending Probe Sets	26
5.4.1. Closure-Driven Extension	26
5.4.2. Replay-Driven Extension.....	26
5.4.3. Coupling-Driven Extension	27
6. IMPLEMENTATION OF XXX	28
6.1. Expression Representation	28
6.1.1. A Hash-Consed n -Ary Circuit DAG	28
6.1.2. Canonicalization: Flattening and Cancellation	28
6.1.3. Multiplicative Factoring.....	29
6.1.3.1. The Factoring Transformation.....	29
6.1.3.2. Enlarging the Class of Verifiable Circuits	29
6.1.3.3. Cost Analysis and the Factoring–Speed Tension	29
6.2. Realizing the Rewrite Engine	29
6.2.1. Reference-Counted Single-Occurrence Discharge.....	30
6.2.2. Recomputing the Find-Rules per Round.....	30
7. EVALUATION.....	31
7.1. Benchmarks	31
7.2. Methodology and Metrics	31
7.3. Results	31
7.3.1. Verification Outcomes and Case Studies	32
7.3.2. Redundancy in the Search	32
7.3.3. The Cost and Benefit of Extending Probe Sets	32
7.4. Discussion	32
8. FUTURE WORK	33
8.1. Arithmetic Circuits on Larger Fields	33
8.2. Symmetry-Orbit Space Exploration	33

8.3. Scaling and Compositional Verification	33
8.4. Glitches and Practical Aspects	34
9. CONCLUSION	35
BIBLIOGRAPHY.....	36
APPENDIX.....	37

LIST OF TABLES

LIST OF FIGURES

LIST OF SYMBOLS

- Φ Coupling function: a family $\{\Phi_{\mathbf{a},\mathbf{b}}\}$ of measure-preserving bijections of the random space, one per ordered pair of secret assignments.
- d Probing order (maximum number of wires an adversary may observe).
- σ Randomness substitution $r \mapsto e \oplus r$; its induced measure-preserving map on the random space is written $\hat{\sigma}$.

LIST OF ABBREVIATIONS

ISW Ishai, Sahai, Wagner.

1. INTRODUCTION

1.1 Side-Channel Attacks and the Masking Countermeasure

Introduce side-channel attacks and masking as a countermeasure.

1.2 Verifying Masked Implementations

Motivate the need for automated verification of masked implementations.

1.3 Scope and Threat Model

State the scope and threat model: ISW probing, software/value-based leakage, glitches out of scope.

1.4 Contributions

List the thesis's contributions: the rewrite engine, factoring, closures, Vandermonde-based subset-space traversal, the coupling view of rewriting, and the profiling/redundancy analysis.

1.5 Thesis Outline

Outline the structure of the remaining chapters.

2. PRELIMINARIES

2.1 Notation and Mathematical Background

Fix general notation used throughout the thesis (sets, tuples, functions, etc.).

2.1.1 The Field $\mathbb{F}_2$ and Multilinear Boolean Polynomials

Recall the field $\mathbb{F}_2$ and the representation of Boolean functions as multilinear polynomials over it.

2.1.2 Probability Distributions and Statistical Independence

Recall the definitions of probability distributions and statistical independence used later to state probing security.

2.1.3 Boolean Circuits and Gates

We now introduce the object the rest of the thesis is built on. The definition is the standard digraph formulation of a circuit, specialised to arithmetic over the

two-element field.

Definition 2.1 (Arithmetic circuit over $\mathbb{F}_2$). *Fix the field $\mathbb{F}_2 = \{0,1\}$ and a set $S = \{s_i\}$ of gate functions, each of some arity $a_i \in \mathbb{N}_{\geq 1}$, i.e. each s_i maps $\mathbb{F}_2^{a_i}$ to $\mathbb{F}_2$. A circuit C is a finite directed graph (a digraph) containing no directed cycle, such that:*

- *Every vertex of indegree 0 is labelled either by an indeterminate, in which case it is an input gate, or by an element of $\mathbb{F}_2$, in which case it is a constant gate. The input gates are partitioned into secret inputs $\mathcal{S}$, random inputs $\mathcal{R}$, and public inputs $\mathcal{P}$.*
- *Every remaining vertex is a computation gate, labelled by a function $s_i \in S$ whose arity a_i coincides with the vertex's indegree. A computation gate of outdegree 0 is an output gate.*

We call the vertices wires, and for a computation gate we speak interchangeably of the gate and the wire carrying its output.

Over $\mathbb{F}_2$ there is no difference between the Boolean and the arithmetic reading of such a circuit: exclusive-or is addition, logical-and is multiplication, and the two together already express every function $\mathbb{F}_2^n \rightarrow \mathbb{F}_2$, i.e. they are universal. So we fix

$$S = \{\oplus, \odot\},$$

addition and multiplication in $\mathbb{F}_2$ (Section 2.1.1), both of arity 2. Every computation gate then has indegree exactly 2 and reads

$$w_i = w_j \oplus w_k \quad \text{or} \quad w_i = w_j \odot w_k.$$

Throughout the thesis $\odot$ denotes multiplication in $\mathbb{F}_2$; we might sometimes use $\cdot$ for the same operation.

Acyclicity is what makes a circuit computable. It lets us list the wires in a topological order $w_1, \dots, w_m$ in which the two operands of every gate appear before the gate itself, so that $j, k < i$. Fixing a value in $\mathbb{F}_2$ for every input gate — a constant gate carries its own label — then determines the value of every wire by induction along this order. When the inputs are clear from context we write simply w_i for this value.

Notation 2.1. *For a wire w_i , $\text{vars}(w_i) \subseteq \mathcal{R}$ is the set of random inputs its defining sub-circuit depends on: $\text{vars}(w_i) = \{w_i\}$ if $w_i \in \mathcal{R}$, $\text{vars}(w_i) = \emptyset$ if $w_i \in \mathcal{S} \cup \mathcal{P}$ or w_i is a constant gate, and $\text{vars}(w_i) = \text{vars}(w_j) \cup \text{vars}(w_k)$ for a gate with operands w_j, w_k .*

Two conventions, fixed here once, are used throughout. First, a topological order realises the circuit as a *straight-line program*: each wire is defined at most once, in terms only of wires already defined, so there is no branching and no recomputation of a gate under a different name. We pass freely between the digraph of Definition 2.1 and this sequential reading, whichever is clearer at the moment. Second, the restriction to 2-ary gates is a modelling choice, not a loss of generality: an n -ary sum or product is always expressible as a chain of 2-ary gates. Forcing this decomposition up front matters for Section 2.3.2, where every gate output is a wire an adversary may separately observe — collapsing several operations into one hidden, unobservable step would understate what a probing adversary can see. Chapter 5 (Section 6.1.1) later represents this same object as a hash-consed data structure suited to computation; nothing here depends on that choice of representation.

2.2 Secret Sharing and Masking

Introduce secret sharing and masking as a side-channel countermeasure.

2.2.1 Boolean Masking

Define Boolean masking (XOR-sharing) of a secret and how masked circuits compute on shares.

2.3 The ISW Probing Model

Introduce the Ishai-Sahai-Wagner (ISW) probing adversary model.

2.3.1 t -Threshold Probing and Observation Sets

Define t -threshold probing and observation sets of wires an adversary may query.

2.3.2 d -Probing Security

Define d -probing security formally, in terms of the joint distribution of any d -wire probe.

2.3.3 Probabilistic Non-Interference

Define probabilistic non-interference (PNI) and relate it to d -probing security.

2.3.4 Threshold and Strong Non-Interference

Define threshold (TNI) and strong non-interference (SNI) and relate them to composability of masked gadgets.

2.4 Randomness Substitution

This section isolates, as a single reusable fact, the probabilistic argument our checker automates. Fix a circuit as in Section 2.1.3, with random inputs $\mathcal{R}$, and let a *randomness assignment* be a function $\rho : \mathcal{R} \rightarrow \{0, 1\}$. The secrets and public inputs are held fixed throughout, so every wire w denotes a function $w(\rho)$ of ρ alone. A

probe is a tuple of wires — the values an adversary observes — and Section 2.3.2 phrases d -probing security as the demand that the joint distribution of a probe of size at most d , taken over the uniformly distributed ρ , not move as the secrets vary.

The whole argument rests on one line of $\mathbb{F}_2$ arithmetic: for any fixed bit c , the translation $b \mapsto c \oplus b$ is a **bijection** of $\{0,1\}$ that preserves the uniform distribution (i.e. it is its own inverse and merely swaps the two outcomes). Everything below is this fact, applied to a single coordinate of ρ with every other coordinate frozen. The object that carries it is a substitution.

Definition 2.2 (Randomness substitution). *Let $r \in \mathcal{R}$ and let e be a wire with $r \notin \text{vars}(e)$. The substitution $\sigma = (r \mapsto e \oplus r)$ acts on a wire w by replacing every occurrence of the input r with the wire $e \oplus r$; we write the resulting wire $w\sigma$. The same σ acts on randomness assignments, through the map $\hat{\sigma} : \{0,1\}^{\mathcal{R}} \rightarrow \{0,1\}^{\mathcal{R}}$ given by*

$$\hat{\sigma}(\rho)(r) = e(\rho) \oplus \rho(r), \quad \hat{\sigma}(\rho)(r') = \rho(r') \quad \text{for } r' \neq r.$$

Geometrically, $\hat{\sigma}$ is a *shear* of the cube $\{0,1\}^{\mathcal{R}}$: it fixes every coordinate but r and offsets that one by $e(\rho)$, a quantity that reads only the frozen coordinates. Restricted to a fibre — a pair of points differing solely in the r -coordinate — it is exactly the translation $\rho(r) \mapsto e(\rho) \oplus \rho(r)$ of the one-line fact above, so what measure it preserves it preserves by translation invariance, fibre by fibre. We call any such map — one coordinate translated by a context depending on the others, everything else left alone — a *shear*, and use the word throughout.

Pairing the two actions under one name is the point of the lemma below: the syntactic rewrite $w \mapsto w\sigma$ and the reparametrisation $\rho \mapsto \hat{\sigma}(\rho)$ are one operation seen from two sides, and $\hat{\sigma}$ does not disturb the distribution.

Lemma 2.1 (Substitution). *For a substitution $\sigma = (r \mapsto e \oplus r)$ as in Definition 2.2, the map $\hat{\sigma}$ is a measure-preserving involution of the uniform distribution on $\{0,1\}^{\mathcal{R}}$, and for every wire w ,*

$$(w\sigma)(\rho) = w(\hat{\sigma}(\rho)) \quad \text{for every randomness assignment } \rho.$$

Proof. Fix every coordinate of ρ other than r . Since $r \notin \text{vars}(e)$, the value $e(\rho)$ depends only on these frozen coordinates, so it is constant as $\rho(r)$ ranges over $\{0,1\}$. On this fibre $\hat{\sigma}$ acts by the translation $\rho(r) \mapsto e(\rho) \oplus \rho(r)$, which is measure-preserving and self-inverse by the observation above, while every other coordinate is left fixed. Hence $\hat{\sigma}$ is an involution of $\{0,1\}^{\mathcal{R}}$ preserving the uniform measure coordinatewise, and so preserving the product measure.

The identity is a structural induction on w . For $w = r$ both sides equal $e(\rho) \oplus \rho(r)$. For an input $w \neq r$ both sides equal $w(\rho)$, since $\hat{\sigma}$ fixes every coordinate but r . For a gate $w_i = w_j \oplus w_k$ or $w_i = w_j \odot w_k$, apply the hypothesis to w_j and w_k and use that $\oplus$ and $\odot$ are computed pointwise from their operands' values. $\square$

Corollary 2.1 (Single-occurrence discharge). *Suppose r occurs exactly once among a probe $\mathbf{w}$ and the wires it depends on, as an operand of a gate $x = e \oplus r$ with $r \notin \text{vars}(e)$, and let $\sigma = (r \mapsto e \oplus r)$. Then $\mathbf{w}\sigma$ is identically distributed to $\mathbf{w}$ over ρ uniform, and every occurrence of x in $\mathbf{w}\sigma$ has collapsed to the bare input r .*

Proof. Distributional equality is Lemma 2.1 applied coordinatewise to $\mathbf{w}$, using that $\hat{\sigma}$ preserves the uniform distribution on $\{0, 1\}^{\mathcal{R}}$. For the second claim, $x\sigma = e \oplus (e \oplus r) = r$ by $\mathbb{F}_2$ -cancellation; since r occurs nowhere else by hypothesis, no other part of $\mathbf{w}$ is touched. $\square$

Remark 2.1. *Rewriting a probe by a substitution — whether in the general form of Lemma 2.1 or the single-occurrence case of Corollary 2.1 — leaves its distribution untouched. That is the key observation that derives the procedure to decide the security of a probe, by one substitution at a time, taking it to a form with no secret in sight. The single-occurrence case is the one the `maskVerif` tool (Barthe et al., 2019) calls optimistic sampling. A derivation is nothing more than such substitutions chained together; how that chain is searched for is taken up in the later chapters.*

2.5 Computational Hardness Preview

Preview the #P-completeness result of Chapter 4, motivating why the thesis pursues a constructive rewrite-based checker rather than exact counting.

3. RELATED WORK

3.1 Language-Based Verification

Survey language-based approaches to masking verification, the main inspiration for this thesis.

3.1.1 Verified Proofs of Higher-Order Masking

Discuss prior work on verified/formal proofs of higher-order masking schemes.

3.1.2 Large Sets and the Exploration Algorithm

Discuss prior large-set exploration algorithms for probing security and how this thesis's subset-space traversal relates.

3.1.3 Witness Replay and Derivation Reuse

Discuss prior work on reusing a certification witness/derivation across related probes, and how it relates to Component III's replay-driven extension (Section 5.4.2).

3.1.4 MaskVerif With Physical Defaults

Discuss `maskVerif` and its physical-default extensions, and compare to the rewrite engine of Chapter 5.

3.2 Fourier-Analytic Verification

Survey Fourier-analytic approaches to masking verification (Bloem et al.).

3.2.1 The Xiao-Massey Criterion and Approximations

Discuss the Xiao-Massey criterion and its approximate variants for testing correlation-immunity/independence.

3.2.2 SAT-Based Realization and Its Cost

Discuss SAT-based realizations of Fourier-analytic verification and their computational cost.

3.3 Other Approaches

Survey other verification approaches not covered above.

3.3.1 Model Counting and SMT

Discuss model-counting and SMT-based approaches to verifying secret-independence.

3.3.2 Compositional Reasoning and Relabeling

Discuss compositional reasoning and relabeling/symmetry techniques used elsewhere, relating to Future Work’s symmetry-orbit proposal.

3.4 Position of This Work

Position this thesis’s contributions relative to the surveyed related work.

4. COUPLINGS AND PROBING SECURITY

4.1 The Counting Problem Is #P-Complete

Introduce the counting problem behind secret-independence checking and state the #P-completeness result.

4.1.1 From Independence to Distribution Counting

Show how deciding secret-independence of a probe reduces to a distribution-counting problem.

4.1.2 #P-Completeness and Its Consequences for Verification

Prove #P-completeness of the counting problem and discuss what this implies for exact verification (motivating the constructive rewrite-based approach instead).

4.2 Couplings of Probabilistic Programs

Introduce couplings as a proof technique for distributional equality between probabilistic programs.

4.2.1 Coupling Two Runs Under Different Secrets

Define a coupling between the run of a probe under one secret assignment and its run under another.

4.2.2 Coupling Functions: Randoms to Randoms

Formalize coupling functions as measure-preserving bijections on the randomness domain.

4.3 Probing Security via Couplings

Relate d -probing security to the existence of a coupling for every probe of size d .

4.3.1 d -Probing Security Implies a (Family of) Coupling Functions

Prove that d -probing security is equivalent to the existence of a family of coupling functions, one per pair of secret assignments.

4.3.2 Secret-Indistinguishability

Connect the coupling-existence characterization back to the standard secret-indistinguishability definition of probing security.

4.4 Substitutions as a Coupling Construction

Explain how the randomness-substitution rewrites of Section 2.4 construct an explicit coupling.

4.4.1 The Derivation of an Explicit Coupling Witness

Show how a derivation (sequence of substitutions) yields an explicit coupling witness for a probe.

4.4.2 Why a Successful Rewrite Yields a Valid Coupling

Prove that a successful rewrite derivation, read as a composition of shears, satisfies the coupling definition.

5. DESIGN OF XXX

5.1 Pipeline Overview

Give a high-level overview of the verification pipeline: the three components (rewrite engine, subset-space traversal, probe-set extension) and how they fit together.

5.2 Component I: The Rewrite Engine

Given a chosen probe $\mathbf{w} = (w_1, \dots, w_k)$, $k \leq d$ — a tuple of wires of the circuit — the rewrite engine decides whether $\mathbf{w}$ is secret-independent. It does so constructively: rather than reasoning about probability distributions directly, it searches for a finite sequence of applications of Lemma 2.1 (Section 2.4) after which $\mathbf{w}$'s image is *syntactically* free of secret leaves — no secret leaf is reachable from any w_i . Termination with a secret-free image certifies $\mathbf{w}$; Theorem 5.1 in Section 5.2.3 makes precise why this certificate is sound.

The engine has three tiers of rewriting rules, increasing in generality and cost, summarized in Algorithm 1 and mirrored in the structure of this section. `SIMPLEREWRITE` (Section 5.2.1) handles the common case, every random occurring at most once, cheaply. It is tried first on the unfactored probe and, on failure, once more after the multiplicative factoring of Section 6.1.3 has had a chance to expose additional single-occurrence randoms. `GENERALREWRITE` (Section 5.2.2) is the complete fallback for the remaining case, a random occurring more than once: it

substitutes a random by its context wherever the random occurs, interleaving factoring lazily whenever both of its own find-rules stall. Section 5.2.3 closes the section by reading the whole derivation as an explicit, replayable bijection on the randomness assignment, the Φ that Component III (Section 5.4) later replays against every other wire in the circuit.

5.2.1 The Simple Rewrite Rule

The cheapest tier discharges the pattern Corollary 2.1 addresses directly: a random that occurs exactly once below the probe, additively — as an operand of an XOR gate. Such a random is a *simple-rule candidate*, provided it is not itself a probe root. Writing $x = e \oplus r$ for its unique parent, the corollary lets us relabel x as a fresh random and drop r without ever looking at the context e : the pattern alone certifies that x , jointly with everything independent of r , is distributed as fresh, independent randomness. The tier applies this rewrite repeatedly, each discharge possibly leaving a further random occurring once, until either no secret leaf is reachable from the probe — success — or no candidate remains — failure.

The exclusion of probe roots is a soundness condition, not an optimisation. A root’s value is what the adversary observes: it is fixed, not fresh, and relabelling it would claim independence for something the tuple actually exposes. The tuple $(r, s \oplus r, r \oplus r_3)$ makes the danger concrete. Its first entry is the bare leaf r , and the first two entries XOR to the secret s , so this tuple leaks s and must be rejected. Yet once r_3 is discharged — it occurs once, additively, in $r \oplus r_3$ — the random r is left occurring only once, in $s \oplus r$. Were roots not guarded, r would now match the candidate pattern and be relabelled away, blinding the very value the adversary reads and reporting a false SECURE. Guarding roots throughout the cascade is what rules this out.

Two properties of the tier matter downstream. First, it fires the shallowest, closest-to-a-root candidates first, matching the increasing-depth discharge order of `maskVerif` (Barthe et al., 2019). Second, a random is discharged at most once — relabelled, it is gone — so the tier performs at most $|\mathcal{R}|$ rewrites and always terminates. Its realisation as an incremental traversal that avoids recomputing the candidate set after each discharge is deferred to Section 6.2.

This tier is sound, since each step is a direct instance of Corollary 2.1, but incom-

Algorithm 1 Discharging a probe (Component I)

```
1: function CHECKPROBE( $\mathbf{w}$ )
Require: probe  $\mathbf{w}$ , a tuple of wires of the circuit
Ensure: SECURE, or INSECURE( $\mathbf{w}$ )
2:   ( $ok, T$ )  $\leftarrow$  SIMPLEREWRITE( $\mathbf{w}$ ) ▷ Tier 1: unfactored
3:   if  $ok$  then return SECURE
4:   end if
5:   ( $ok, T$ )  $\leftarrow$  SIMPLEREWRITE(Factor( $\mathbf{w}$ )) ▷ Tier 2: factor (§6.1.3), retry
6:   if  $ok$  then return SECURE
7:   end if
8:   ( $ok, T$ )  $\leftarrow$  GENERALREWRITE( $\mathbf{w}, \emptyset, fuel, false, \langle \rangle$ ) ▷ Tier 3: complete
   fallback
9:   return  $ok ? \text{SECURE} : \text{INSECURE}(\mathbf{w})$ 
10: end function

11: function SIMPLEREWRITE( $roots$ )
12:    $todo \leftarrow \{r \in \text{vars}(roots) : r \text{ occurs once, additively, } r \notin roots\}$ , shallowest first
13:    $T \leftarrow \langle \rangle$ 
14:   while  $todo \neq \langle \rangle$  and not SECRETFREE( $roots$ ) do
15:      $r \leftarrow$  pop shallowest element of  $todo$ 
16:      $x \leftarrow$  unique additive parent of  $r$  ▷  $x = e \oplus r$ 
17:     mark  $x$  as a fresh random leaf
18:      $T \leftarrow T.\text{push}(r, x)$  ▷  $t = e \oplus r = x$ , already in hand
19:     drop  $r$  from the tuple
20:     (a random newly occurring once, additively, and  $\notin roots$ , joins  $todo$ )
21:   end while
22:   return (SECFREE( $roots$ ),  $T$ ) ▷ same shears, now recorded — §5.2.3
23: end function

24: function GENERALREWRITE( $tuple, used, fuel, factored, T$ )
25:   if SECRETFREE( $tuple$ ) then return ( $true, T$ )
26:   end if
27:   if  $fuel = 0$  then return ( $false, T$ )
28:   end if
29:    $r \leftarrow$  FINDSIMPLE( $tuple$ ) ▷ occurs once, additively, not a root
30:   if  $r = \perp$  then  $r \leftarrow$  FINDGENERAL( $tuple, used$ ) ▷ occurs  $\geq 2\times$ ; some
   additive parent free of  $r$ 
31:   end if
32:   if  $r = \perp$  then
33:     if  $factored$  then return ( $false, T$ )
34:     end if
35:     return GENERALREWRITE(Factor( $tuple$ ),  $used, fuel, true, T$ ) ▷ §6.1.3.3,
   lazy retry
36:   end if
37:    $x \leftarrow$  chosen additive parent of  $r$ ;  $e \leftarrow \bigoplus(\text{children}(x) \setminus \{r\})$  ▷ side cond.
    $r \notin \text{vars}(e)$ 
38:    $t \leftarrow e \oplus r$ ;  $tuple' \leftarrow tuple[r \mapsto t]$  ▷ substitute everywhere — Lemma 2.1
39:   return GENERALREWRITE( $tuple', used \cup \{r\}, fuel - 1, false, T.\text{push}(r, t)$ )
40: end function
```

plete: it has nothing to offer a random occurring twice, such as a mask reused across two cross-terms of a masked multiplication. Section 5.2.2’s fallback handles that case.

5.2.2 Fallback: General Substitution

GENERALREWRITE handles randoms that occur more than once by performing the substitution of Lemma 2.1 literally, rather than only its single-occurrence case (Corollary 2.1). It picks a random r and an additive occurrence $x_1 = e \oplus r$ with $r \notin \text{vars}(e)$, then rewrites every root by substituting $r \mapsto e \oplus r$ throughout. Cancellation in $\mathbb{F}_2$ collapses the chosen occurrence x_1 to the bare leaf r , exactly as in Corollary 2.1, while every other occurrence of r absorbs e instead of vanishing, cancelling further if it happens to share structure with e . The loop succeeds once no secret leaf remains reachable from the rewritten roots.

Each round re-examines the current tuple for an applicable substitution. FINDSIMPLE looks first for the same single-occurrence, non-root pattern as Section 5.2.1, since a general substitution can turn a multi-occurrence random into a single-occurrence one; it prefers the shallowest candidate. Failing that, FINDGENERAL looks for any random, not yet used by a previous general step, occurring at least twice with some additive parent x_1 whose other children do not themselves depend on it — the side condition $r \notin \text{vars}(e)$ that Lemma 2.1 requires. It prefers the deepest candidate, the opposite bias from FINDSIMPLE; no deeper principle motivates this choice beyond the ordering itself. A context e is always read off an XOR gate’s other children: anything else would mean e contains r , making $r \mapsto e \oplus r$ fail to be a bijection, so such a context is rejected outright rather than admitted as an unsound relabelling.

When both find-rules stall on the tuple’s current form, the engine factors (Section 6.1.3) once and retries, rather than declaring failure immediately. Factoring can move a random from a purely multiplicative position, where neither find-rule can see it, into an additive one, but on its own it eliminates no secret, so it must be interleaved with substitution rather than run once up front or once at the end. Factoring is symmetrically not tried before the simple rule either: Algorithm 1 always tries SIMPLEREWRITE unfactored first, because factoring can destroy a single-occurrence opportunity the unfactored form exposes, so trying the cheap, unfactored tier first is not just faster on the common case but strictly more complete.

FINDGENERAL is needed precisely when a random occurs additively in two places that do not telescope into one another under the simple rule. Suppose r occurs as an operand of $x_1 = e_1 \oplus r$ in one root and of $x_2 = e_2 \oplus r$ in another, with $r \notin \text{vars}(e_1) \cup \text{vars}(e_2)$. Substituting through x_1 , i.e. $r \mapsto e_1 \oplus r$, turns x_1 into $e_1 \oplus (e_1 \oplus r) = r$ by cancellation, as in Corollary 2.1, while the same substitution turns x_2 into $e_2 \oplus e_1 \oplus r$. If $e_1 \oplus e_2$ is itself secret-free, both roots are secret-free after this one step; otherwise the loop continues, substituting further or factoring to expose a new opportunity. This is the shape of the case a masked multiplication typically produces, where a single random masks two different cross-terms: neither term's random is disposable on its own, but the substitution above can still cancel the secret-dependent part they share. Whether a given instance is discharged this way, and in how many rounds, depends entirely on how much of $e_1 \oplus e_2$ the rest of the derivation can still cancel; the loop offers no guarantee beyond trying. Even granting unlimited fuel, it only composes single-coordinate shears $\hat{\sigma}$, so it can only certify secret-independence witnessed by a chain of substitutions, possibly interleaved with factoring. Completeness is proved only in the linear/affine case (nerede proof?); whether a genuinely irreducible counterexample exists beyond it is not settled by this thesis.

Termination is only argued informally, not proved. Each general substitution retires its random into *used* and is never repeated for it, bounding the number of general steps by $|\mathcal{R}|$; between general steps, simple substitutions strictly reduce the candidate pool; factoring is retried at most once per stall before the engine gives up on the current form. Interleaving factoring with two mutually triggering find-rules resists a clean potential-function argument, so a generous fuel counter backstops the loop instead. Exhausting fuel is treated as failure, which is sound (Theorem 5.1 is never invoked on a fuel-out) but not necessarily tight: a fuel-out reports the probe as though it were genuinely insecure, without exhibiting an actual distinguishing observation.

5.2.3 Relation to the Coupling-Construction View

A discharge does more than answer SECURE/INSECURE. Read the right way, it hands back the object Chapter 4 asks for: a coupling for the probe. We state that object declaratively first, then show how the engine produces one.

Let the secrets range now, rather than staying fixed as in Section 2.4: write $\mathbf{w}(\rho; \mathbf{a})$

for the value of the probe under randomness assignment $\rho \in \{0,1\}^{\mathcal{R}}$ and secret assignment $\mathbf{a} \in \{0,1\}^{\mathcal{S}}$, the public inputs held fixed.

Definition 5.1 (Coupling of a probe). *A coupling of a probe $\mathbf{w}$ is a family $\{\Phi_{\mathbf{a},\mathbf{b}}\}_{\mathbf{a},\mathbf{b} \in \{0,1\}^{\mathcal{S}}}$ of measure-preserving bijections of $\{0,1\}^{\mathcal{R}}$, one per ordered pair of secret assignments, satisfying*

$$\mathbf{w}(\rho; \mathbf{a}) = \mathbf{w}(\Phi_{\mathbf{a},\mathbf{b}}(\rho); \mathbf{b}) \quad \text{for every } \rho \in \{0,1\}^{\mathcal{R}}.$$

Definition 5.1 says precisely where to send the randomness so that a run under $\mathbf{a}$ is reproduced, observation for observation, by a run under $\mathbf{b}$: each $\Phi_{\mathbf{a},\mathbf{b}}$ dictates the relabelling of the random tape that absorbs the change of secret. Since it is measure-preserving, pushing the uniform tape through it turns this pointwise identity into an equality of laws, so the distribution of $\mathbf{w}(\cdot; \mathbf{a})$ matches that of $\mathbf{w}(\cdot; \mathbf{b})$. A coupling existing for *every* pair $(\mathbf{a}, \mathbf{b})$ is thus the same statement as $\mathbf{w}$ being secret-independent — which is what makes it the right certificate. The family carries the structure one expects of such a certificate: one may take $\Phi_{\mathbf{a},\mathbf{a}} = \text{id}$, $\Phi_{\mathbf{b},\mathbf{a}} = \Phi_{\mathbf{a},\mathbf{b}}^{-1}$, and $\Phi_{\mathbf{b},\mathbf{c}} \circ \Phi_{\mathbf{a},\mathbf{b}} = \Phi_{\mathbf{a},\mathbf{c}}$, so a single anchor $\Phi_{\mathbf{a}_0,\cdot}$ already generates all of it. It is these properties we care about, not any one procedure that realises them.

What the engine records is one such procedure. Discharging a probe amounts to finding a *derivation*: a finite sequence of rewrites

$$\sigma_1 = (r_1 \mapsto e_1 \oplus r_1), \dots, \sigma_K = (r_K \mapsto e_K \oplus r_K),$$

each valid — $r_i \notin \text{vars}(e_i)$ — against the tuple’s state just before step i , after which no secret leaf remains reachable. A rewrite is a rewrite: whatever rule proposed it, and whether the random it retires occurred once or many times, each σ_i is a substitution in the sense of Definition 2.2, hence a single measure-preserving shear $\widehat{\sigma}_i$ (Lemma 2.1). The coupling is assembled from these shears and from nothing else, so the construction below never needs to know how any individual rewrite was found.

Theorem 5.1 (Soundness of discharge). *If $\text{CHECKPROBE}(\mathbf{w})$ (Algorithm 1) returns SECURE, then the joint distribution of $\mathbf{w}(\rho; \mathbf{a})$, for ρ sampled uniformly at random, does not depend on the secret assignment $\mathbf{a}$. Moreover the recorded substitutions realise a coupling of $\mathbf{w}$ in the sense of Definition 5.1.*

Proof sketch. Fix a secret assignment $\mathbf{a}$. Each recorded step contributes the tape shear $\widehat{\sigma}_i$, whose defining context e_i may itself read the secrets, so write $\widehat{\sigma}_i^{\mathbf{a}}$ for its

action under $\mathbf{a}$ and set

$$\varphi_{\mathbf{a}} = \widehat{\sigma_1}^{\mathbf{a}} \circ \widehat{\sigma_2}^{\mathbf{a}} \circ \dots \circ \widehat{\sigma_K}^{\mathbf{a}},$$

a composition of measure-preserving bijections, hence itself one. Writing $\mathbf{w}_K = \mathbf{w}\sigma_1 \dots \sigma_K$ for the final tuple and chaining Lemma 2.1 through the K steps gives $\mathbf{w}(\varphi_{\mathbf{a}}(\rho); \mathbf{a}) = \mathbf{w}_K(\rho)$ for every ρ . A SECURE verdict means $\mathbf{w}_K$ is syntactically secret-free — no secret leaf is reachable from it — so $\mathbf{w}_K(\rho)$ does not depend on $\mathbf{a}$. The right-hand side is therefore common to all secret assignments, and for any two, $\mathbf{a}$ and $\mathbf{b}$,

$$\mathbf{w}(\varphi_{\mathbf{a}}(\rho); \mathbf{a}) = \mathbf{w}_K(\rho) = \mathbf{w}(\varphi_{\mathbf{b}}(\rho); \mathbf{b}).$$

Substituting $\rho \mapsto \varphi_{\mathbf{a}}^{-1}(\rho)$ and setting $\Phi_{\mathbf{a}, \mathbf{b}} = \varphi_{\mathbf{b}} \circ \varphi_{\mathbf{a}}^{-1}$ gives $\mathbf{w}(\rho; \mathbf{a}) = \mathbf{w}(\Phi_{\mathbf{a}, \mathbf{b}}(\rho); \mathbf{b})$ for all ρ ; the family $\{\Phi_{\mathbf{a}, \mathbf{b}}\}$ is the promised coupling, and measure-preservation of each member yields the distributional claim. The chaining uses only that each σ_i is a substitution (Definition 2.2); a single-occurrence rewrite, where e_i is read off one $\oplus$ -parent as in Corollary 2.1, is folded in exactly like any other, so the argument is indifferent to how the derivation was produced. $\square$

The composite $\varphi_{\mathbf{a}}$ is displayed above as a product of shears only for exhibition; it is never used. Component III (Section 5.4.3) uses a single member of the family as a *blinding certificate*: it takes any other wire u in the circuit and admits it into the certified *safe set* when $u \circ \varphi_{\mathbf{a}}$ is secret-free — when the same relabelling that blinds the probe blinds u too. Evaluating $\varphi_{\mathbf{a}}$ on a sampled tape is a back-substitution through the recorded steps in reverse order $i = K, \dots, 1$, since the net image of r_i is $t_i = e_i \oplus r_i$ evaluated against the assignment already updated by every later step: the derivation performs forward elimination, and reading the net map off is back-substitution through the same steps in reverse.

Keeping this record costs nothing. Each rewrite already names its target $t_i = e_i \oplus r_i$ — the very wire it substitutes for r_i — so the derivation $\sigma_1, \dots, \sigma_K$ can be accumulated as the rewrites happen, and the coupling is read off that one list. Component III replays these shears directly against every other wire in the circuit; no probe and no rewrite is special-cased, and each step that helped certify the probe is a step that can help blind its neighbours.

5.3 Component II: Traversing the Subset Space

d -probing security asks that *every* set of at most d wires be secret-independent (Section 2.3.2). Read literally, that is $\sum_{i=0}^d \binom{N}{i}$ separate questions for a circuit of N wires. Component II is the bookkeeping that keeps the question exact while never enumerating a subset redundantly.

Two elementary observations are helpful to understand the traversal. First, we should see that secret-independence passes to subsets, specifically a subset of a secret-independent set is a marginal of a secret-independent law, hence itself is secret-independent. Certifying a set of wires therefore certifies all of its subsets in one stroke. Second, it is enough to look at probes of size *exactly* d : a smaller probe is a marginal of some size- d probe (whenever $N \geq d$, which always holds here), so it is covered once the size- d probes are. The task is thus to certify every d -element subset of the wire set, in batches as large as possible rather than one at a time.

5.3.1 The Representation of the Subset Space

The traversal never holds an explicit list of subsets; it holds a compact description of a whole family of them at once. A *factor* is a pair (c, W) of a count $c \in \mathbb{N}$ and a wire set W , standing for the family $\binom{W}{c}$ of all c -element subsets of W . A *worklist* is a finite sequence of factors

$$wl = \langle (c_1, W_1), \dots, (c_m, W_m) \rangle, \quad W_1, \dots, W_m \text{ pairwise disjoint},$$

denoting the probes obtained by drawing c_j wires from each W_j and taking the union:

$$\llbracket wl \rrbracket = \left\{ S_1 \cup \dots \cup S_m : S_j \in \binom{W_j}{c_j} \right\}, \quad |\llbracket wl \rrbracket| = \prod_{j=1}^m \binom{|W_j|}{c_j}.$$

Disjointness of the W_j makes that union a disjoint one, so every probe has size $\sum_j c_j$ and none is described twice. Two vacuous worklists bound the recursion: if some factor asks for more than it holds, $|W_j| < c_j$, then $\binom{W_j}{c_j} = \emptyset$ and $\llbracket wl \rrbracket$ is empty; if every $c_j = 0$, the sole member is the empty probe. Either way there is nothing to certify. The verification task itself is the single-factor worklist $\langle (d, \mathcal{W}) \rangle$, whose

denotation is $\binom{\mathcal{W}}{d}$, i.e. all d -subsets of the circuit's wires $\mathcal{W}$.

Algorithm 2 sweeps this representation with three moves: discharge the union of the whole worklist and, on success, prune the branch; otherwise certify a representative probe and extend it to a safe set; then split each factor about that safe set and recurse on the pieces. The next two subsections treat the moves prune-and-extend and split, and presents a proof that the split loses nothing.

Algorithm 2 Traversing the subset space (Component II)

```

1: function CHECKALL( $wl = \langle (c_1, W_1), \dots, (c_m, W_m) \rangle$ )
2:   if  $\llbracket wl \rrbracket = \emptyset$  then return SECURE ▷ some  $|W_j| < c_j$ , or every  $c_j = 0$ 
3:   end if
4:    $U \leftarrow W_1 \cup \dots \cup W_m$ 
5:   if CHECKPROBE( $U$ ) = SECURE then return SECURE ▷ large-set pruning (§5.3.2)
6:   end if
7:    $\mathbf{w} \leftarrow$  a representative of  $\llbracket wl \rrbracket$  ▷  $c_j$  wires from each  $W_j$ 
8:    $(sec, T) \leftarrow$  CHECKPROBE( $\mathbf{w}$ ) ▷ Component I (§5.2)
9:   if not  $sec$  then return INSECURE( $\mathbf{w}$ )
10:  end if
11:   $\mathbf{safe} \leftarrow$  EXTEND( $\mathbf{w}, T$ ) ▷ Component III (§5.4);  $\mathbf{w} \subseteq \mathbf{safe}$ 
12:  for all  $(i_1, \dots, i_m) \in \prod_j \{0, \dots, c_j\}$  with  $(i_j)_j \neq (c_j)_j$  do
13:     $wl' \leftarrow \langle (i_j, W_j \cap \mathbf{safe}), (c_j - i_j, W_j \setminus \mathbf{safe}) \rangle_{j=1}^m$  ▷ drop zero-count parts
14:     $r \leftarrow$  CHECKALL( $wl'$ )
15:    if  $r \neq$  SECURE then return  $r$ 
16:    end if
17:  end for
18:  return SECURE
19: end function

```

5.3.2 Large-Set Pruning

Before splitting anything, the traversal tries to discharge the whole batch at once. Let $U = W_1 \cup \dots \cup W_m$ be every wire the worklist can reach, and hand U — as a single tuple — to the rewrite engine of Component I. If U is certified secret-independent, so is every subset of U , and in particular every probe in $\llbracket wl \rrbracket$; the branch is secure and the traversal returns without descending. On some secure circuits this one test does almost all of the work — most branches never split, because their union already discharges.

However this discharge may fail, and it is worth seeing why the algorithm cannot

stop here on failure. The union U is typically far larger than d , and a large tuple can be secret-*dependent* while every one of its size- d sub-tuples is secret-independent — so failing to discharge U says nothing about the individual probes. When the union fails, we descend. We pick a *representative* $\mathbf{w} \in \llbracket wl \rrbracket$ — one probe of the target size, c_j wires from each factor — certify it with Component I, and, if it holds, grow it with Component III (Section 5.4) into a *safe set*

$$\mathbf{safe} \supseteq \mathbf{w}, \quad \text{every subset of which is secret-independent.}$$

This will be further explained in Component III. Put briefly, one measure-preserving coupling blinds every wire of $\mathbf{safe}$ simultaneously, so any sub-tuple of $\mathbf{safe}$, of any size, is secret-independent. Because the representative draws at least one wire from every factor, $\mathbf{safe}$ meets every W_j . This single fact is what makes the split below both exhaustive and strictly progressing.

5.3.3 Completeness via Vandermonde's Identity

With a safe set in hand the split is forced by a counting identity. Fix $\mathbf{safe}$ and cut each factor's ground set along it,

$$W_j = A_j \uplus B_j, \quad A_j = W_j \cap \mathbf{safe}, \quad B_j = W_j \setminus \mathbf{safe}.$$

Lemma 5.1 (Set Vandermonde). *For disjoint wire sets A, B and any $c \geq 0$,*

$$\binom{A \uplus B}{c} = \biguplus_{i=0}^c \left\{ S_A \cup S_B : S_A \in \binom{A}{i}, S_B \in \binom{B}{c-i} \right\},$$

a disjoint union: each c -subset S lies in exactly the block $i = |S \cap A|$. Counting both sides recovers Vandermonde's identity, $\binom{|A|+|B|}{c} = \sum_{i=0}^c \binom{|A|}{i} \binom{|B|}{c-i}$.

The lemma is immediate, a c -subset is determined by its safe part and its unsafe part, chosen independently. Stating it set-wise, and not merely as the numeric identity, is the point: it says not only *how many* probes each block holds but exactly *which*. Applying it in every factor and multiplying out,

$$\llbracket wl \rrbracket = \prod_{j=1}^m \binom{W_j}{c_j} = \biguplus_{(i_1, \dots, i_m)} \llbracket wl_{(i_j)} \rrbracket, \quad wl_{(i_j)} = \left\langle (i_j, A_j), (c_j - i_j, B_j) \right\rangle_{j=1}^m,$$

one *cell* for each distribution $(i_1, \dots, i_m) \in \prod_j \{0, \dots, c_j\}$ recording, factor by factor,

how many of a probe's wires come from the safe side. The cells are disjoint and exhaust $\llbracket wl \rrbracket$, since every probe declares in each factor exactly how much of it is safe and each cell is again the denotation of a worklist, on which the traversal recurses directly.

One cell needs no recursion. The *all-safe* cell $(i_j)_j = (c_j)_j$ collects the probes drawn entirely from the safe sides $A_j \subseteq \mathbf{safe}$ — the probes lying inside $\mathbf{safe}$ — and every one of those is secret-independent by the guarantee of Component III. The algorithm skips it and recurses on the rest. Skipping that particular cell means, with each recursing we are strictly decreasing the amount of probe sets we have to consider, which gives us our termination guarantee.

Theorem 5.2 (Soundness of the traversal). *CHECKALL(wl) (Algorithm 2) terminates, and if it returns SECURE then every probe in $\llbracket wl \rrbracket$ is secret-independent. In particular CHECKALL $\langle (d, \mathcal{W}) \rangle$, returning SECURE, certifies d -probing security.*

Proof. Termination. Order worklists by the multiset $\{|W_1|, \dots, |W_m|\}$ of their ground-set sizes, under the multiset order. Take any cell the algorithm recurses on: it is neither the all-safe cell nor empty, so some factor j^* has $i_{j^*} < c_{j^*}$, forcing $B_{j^*} \neq \emptyset$; and $A_{j^*} \supseteq \mathbf{w} \cap W_{j^*} \neq \emptyset$ because the representative meets every factor. So W_{j^*} is replaced by the two strictly smaller ground sets A_{j^*}, B_{j^*} , while no other factor's ground set grows ($A_j, B_j \subseteq W_j$). The multiset strictly decreases, so the recursion is well-founded.

Soundness, by induction along that order. CHECKALL returns SECURE only in one of three ways. If the union $U = \bigcup_j W_j$ discharges, every subset of U — hence all of $\llbracket wl \rrbracket$ — is secret-independent by marginalisation. If a degenerate worklist bottoms out, $\llbracket wl \rrbracket$ is empty or the lone empty probe, trivially secure. Otherwise the representative certifies and the recursive cells all return SECURE: by Lemma 5.1 those cells together with the skipped all-safe cell partition $\llbracket wl \rrbracket$; the all-safe cell is secret-independent by Component III, and each recursive cell is $\llbracket wl_{(i_j)} \rrbracket$ of strictly smaller measure, secret-independent by the induction hypothesis. Every probe of $\llbracket wl \rrbracket$ lies in exactly one part, so all are secret-independent. Finally $\llbracket \langle (d, \mathcal{W}) \rangle \rrbracket = \binom{\mathcal{W}}{d}$, and by the size reduction of the section opening a SECURE verdict there certifies d -probing security. $\square$

We close this section with two remarks. The soundness above is one-directional by design: a SECURE verdict is trustworthy, but the traversal inherits Component I's incompleteness verbatim and nothing more. An INSECURE verdict is raised only when Component I declines a representative (Section 5.2.2), never because a subset

slipped through unexamined — Lemma 5.1 guarantees the cells cover $\llbracket wl \rrbracket$ exactly, so the sweep is genuinely exhaustive. The reported witness is therefore a true counterexample precisely when Component I is complete on it, as in the linear/affine case; a fuel-bounded refusal there is the one place a spurious INSECURE can enter, and it enters through Component I, not through this traversal. Second, the efficiency of the whole scheme rests on the safe set being large: the bigger **safe**, the fewer and smaller the surviving cells, and the closer the union prune comes to settling a branch outright. That is exactly the quantity Component III’s coupling extension is built to maximise, and it is where the approach of this thesis departs from a plain set-splitting search.

5.4 Component III: Extending Probe Sets

Introduce the idea of extending a certified probe to certify additional wires cheaply, via closures, replay, and couplings.

5.4.1 Closure-Driven Extension

Describe extension via closure: wires already implied secret-independent by set-theoretic closure of certified probes.

5.4.2 Replay-Driven Extension

Describe extension by replaying a certified probe’s rewrite derivation against a new wire.

5.4.3 Coupling-Driven Extension

Describe extension via the coupling/blinding-certificate view of Section 5.2.3: admitting a wire u when $u \circ \varphi_{\mathbf{a}}$ is secret-free.

6. IMPLEMENTATION OF XXX

6.1 Expression Representation

Introduce the concrete data structure used to represent circuit expressions internally.

6.1.1 A Hash-Consed n -Ary Circuit DAG

Describe the hash-consed n -ary DAG representation of circuits and why hash-consing is used (sharing, fast equality).

6.1.2 Canonicalization: Flattening and Cancellation

Describe the canonicalization pass: flattening nested XORs and cancelling repeated operands in $\mathbb{F}_2$.

6.1.3 Multiplicative Factoring

Introduce multiplicative factoring as a transformation that exposes additional single-occurrence randoms.

6.1.3.1 The Factoring Transformation

Give the precise factoring transformation and an example of it exposing a hidden single-occurrence random.

6.1.3.2 Enlarging the Class of Verifiable Circuits

Show how factoring enlarges the class of circuits the rewrite engine can certify.

6.1.3.3 Cost Analysis and the Factoring–Speed Tension

Analyze the computational cost of factoring and the resulting tension with verification speed, motivating the lazy interleaving used in Section 5.2.2.

6.2 Realizing the Rewrite Engine

Describe the concrete implementation of Algorithm 1 (Section 5.2).

6.2.1 Reference-Counted Single-Occurrence Discharge

Describe the reference-counting mechanism used to find single-occurrence randoms in SIMPLEREWRITE without recomputation.

6.2.2 Recomputing the Find-Rules per Round

Describe the implementation of FINDSIMPLE/FINDGENERAL and how they are (re)computed each round.

7. EVALUATION

7.1 Benchmarks

Describe the benchmark suite of masked circuits used to evaluate the tool (sources, sizes, masking orders).

7.2 Methodology and Metrics

Describe the evaluation methodology and the metrics reported (verification time, number of probes discharged, redundancy, etc.).

7.3 Results

Present the overall results of running the tool on the benchmark suite.

7.3.1 Verification Outcomes and Case Studies

Report verification outcomes per benchmark, including the case study of the insecure 4-share $x + yz$ finding.

7.3.2 Redundancy in the Search

Analyze redundancy in the search: union vs. chosen re-discharge, and closure collisions.

7.3.3 The Cost and Benefit of Extending Probe Sets

Measure the cost/benefit trade-off of Component III's probe-set extension against the baseline traversal.

7.4 Discussion

Discuss what the results imply about the tool's strengths, limitations, and how it compares to related work.

8. FUTURE WORK

8.1 Arithmetic Circuits on Larger Fields

Discuss extending the approach beyond $\mathbb{F}_2$ to larger fields.

8.2 Symmetry-Orbit Space Exploration

Discuss exploiting circuit automorphisms/relabeling symmetries to reduce the subset space explored.

8.3 Scaling and Compositional Verification

Discuss compositional verification techniques for scaling to larger circuits/gadgets.

8.4 Glitches and Practical Aspects

Discuss extending the model to account for glitches and other practical hardware effects.

9. CONCLUSION

Write the conclusion: summarize the problem, the approach (rewrite engine, subset-space traversal, probe-set extension), the main results, and their significance.

BIBLIOGRAPHY

- Barthe, G., Belaïd, S., Cassiers, G., Fouque, P.-A., Grégoire, B., & Standaert, F.-X. (2019). Maskverif: Automated verification of higher-order masking in presence of physical defaults. In K. Sako, S. A. Schneider, & P. Y. A. Ryan (Eds.), *Computer security – esorics 2019* (pp. 300–318, Vol. 11735). Springer International Publishing. https://doi.org/10.1007/978-3-030-29959-0_15

APPENDIX

Add supplementary experimental data (extra tables, plots, benchmark details) supporting the Evaluation chapter.